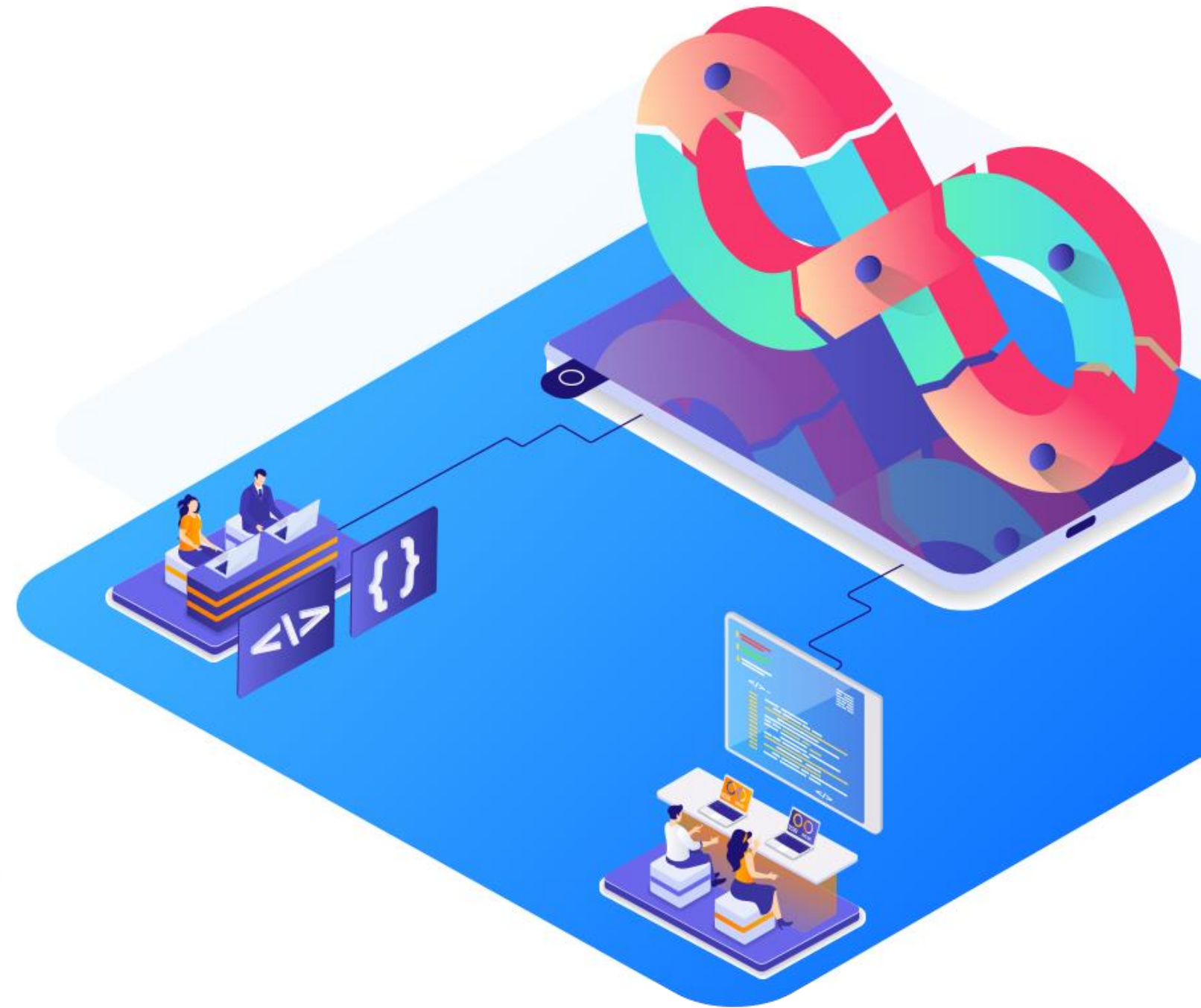


**DevSecOps**



# Conceptualizing DevSecOps and Its Principles



# Learning Objectives

By the end of this lesson, you will be able to:

- 🔗 Implement DevSecOps practices to integrate security throughout the software development lifecycle ensuring robust, secure software outputs
- 🔗 Utilize threat modeling techniques such as STRIDE, DREAD, and PASTA to analyze and categorize threats for effectively anticipating and mitigating potential security breaches
- 🔗 Implement security controls and testing throughout each phase of the DevSecOps pipeline to ensure a continuous defense posture during application development

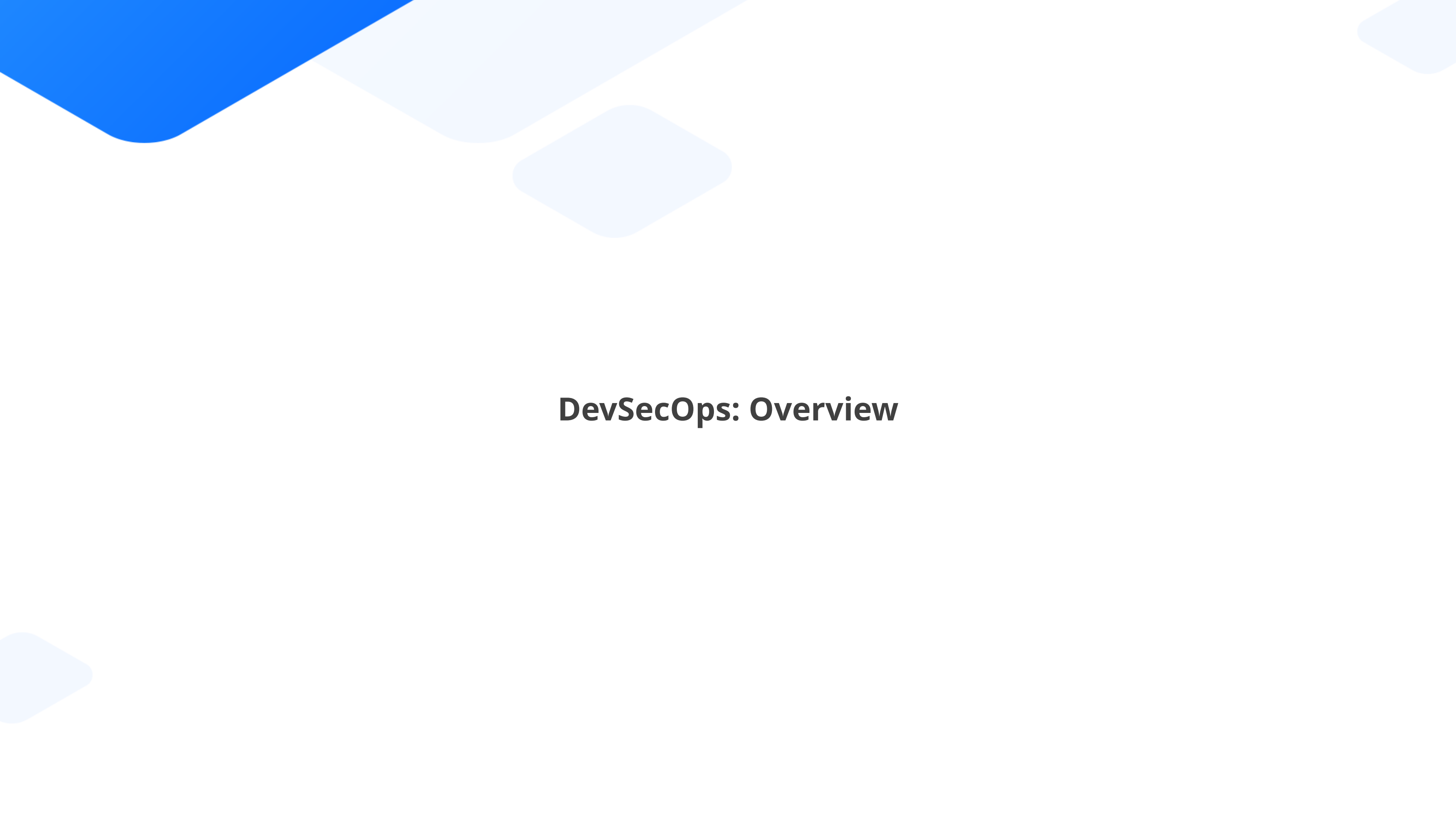


# Learning Objectives

By the end of this lesson, you will be able to:

- Execute secure coding practices to minimize vulnerabilities in software applications for enhancing the security and reliability of software products
- Utilize strategies and tools for debugging security issues in CI/CD pipelines for identifying and resolving security weaknesses swiftly and efficiently

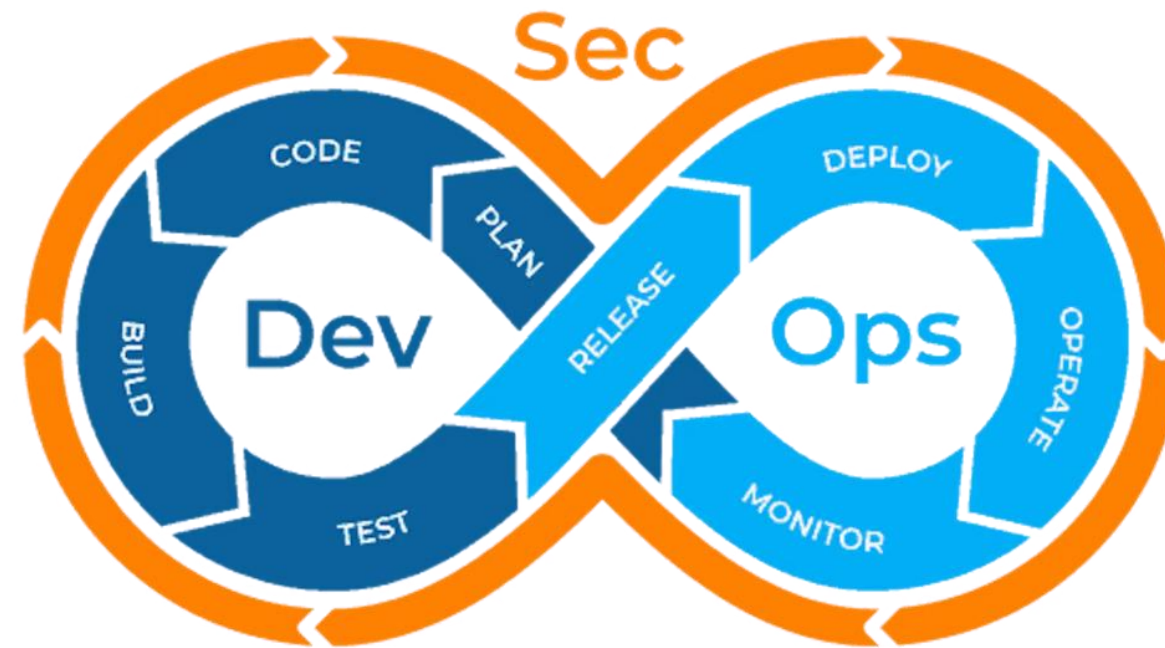




# DevSecOps: Overview

# What Is DevSecOps?

DevSecOps, which stands for development, security, and operations, is a practice that integrates security into all phases of the software development lifecycle.



It includes tools and processes that encourage collaboration between developers, security specialists, and operation teams to build software that is both efficient and secure.

# Importance of DevSecOps

DevSecOps actively enhances security and mitigates risks in fast-paced development environments, ensuring comprehensive protection across all stages of software creation.

## Enhanced security in fast-paced development

It seamlessly integrates security into the rapid development and deployment cycles of modern software engineering.



## Protection against software vulnerabilities

It is pivotal in safeguarding organizations from the risks posed by software vulnerabilities.

# Benefits of DevSecOps

## **Proactive threat mitigation**

It addresses security threats early, preventing vulnerabilities from reaching production.

## **Fast development alignment**

It keeps security in sync with rapid development cycles, avoiding delays.

## **Enhanced collaboration**

It improves communication between development, security, and operations teams for unified software delivery.

## **Cost efficiency**

It lowers costs by resolving security issues early, reducing the need for costly fixes post-deployment.

# DevSecOps Best Practices

Organizations use the following approaches to support digital transformation with DevSecOps:

Shift left

It involves identifying and addressing vulnerabilities early in the development process to ensure secure code from the start.

Shift right

It focuses on security after deployment, catching issues that may arise during real-world use.

Use automated security tools

It integrates security scans into CI/CD pipelines to maintain development speed without compromising security.

Promote security awareness

It makes security a shared responsibility, with every team member accountable for protecting users.

# DevSecOps Tools

They help identify and fix security issues throughout the software development lifecycle, ensuring that security is integrated at every stage.

Some common tools include:

- **Static Application Security Testing (SAST) tools:** It analyzes and detects vulnerabilities in proprietary source code before execution.
- **Software Composition Analysis (SCA) tools:** It automates visibility into open-source software (OSS) for risk management and security and license compliance.
- **Interactive Application Security Testing (IAST) tools:** It evaluates an application's vulnerabilities in a production environment using security monitors running within the application.
- **Dynamic Application Security Testing (DAST) tools:** It mimics hackers by testing the application's security from outside the network.

## Quick Check

A company is adopting DecSecOps practices to improve security in their software development process. They aim to catch vulnerabilities early, enhance collaboration, and automate security assessments. Which of the following actions best reflects a key DevSecOps best practice?

- A. Performing security checks after software is fully developed
- B. Incorporating security testing throughout development using tools like SAST and DAST
- C. Keeping the development, security, and operations teams working separately
- D. Avoiding the use of automated security tools to focus on manual assessments





# Threat Modeling Fundamentals

# What Is Threat Modeling?

It is a proactive approach to identifying potential attack vectors by assessing all possible risks across an entire application, ensuring comprehensive security coverage.



# Importance of Threat Modeling in DevSecOps

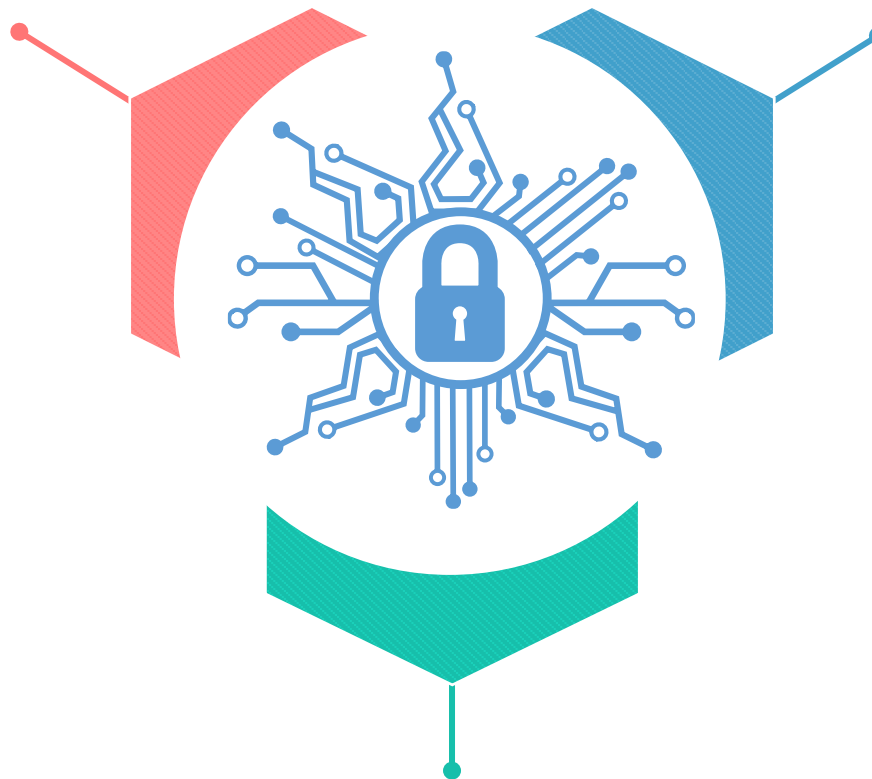
It is vital for detecting and responding to cyber threats early, protecting sensitive data, and enhancing long-term security strategies to safeguard against evolving risks.

## Early detection of security risks

It identifies vulnerabilities during design, saving time and cost in later stages

## Protecting sensitive data

It proactively secures data to prevent breaches.



## Strategic risk mitigation

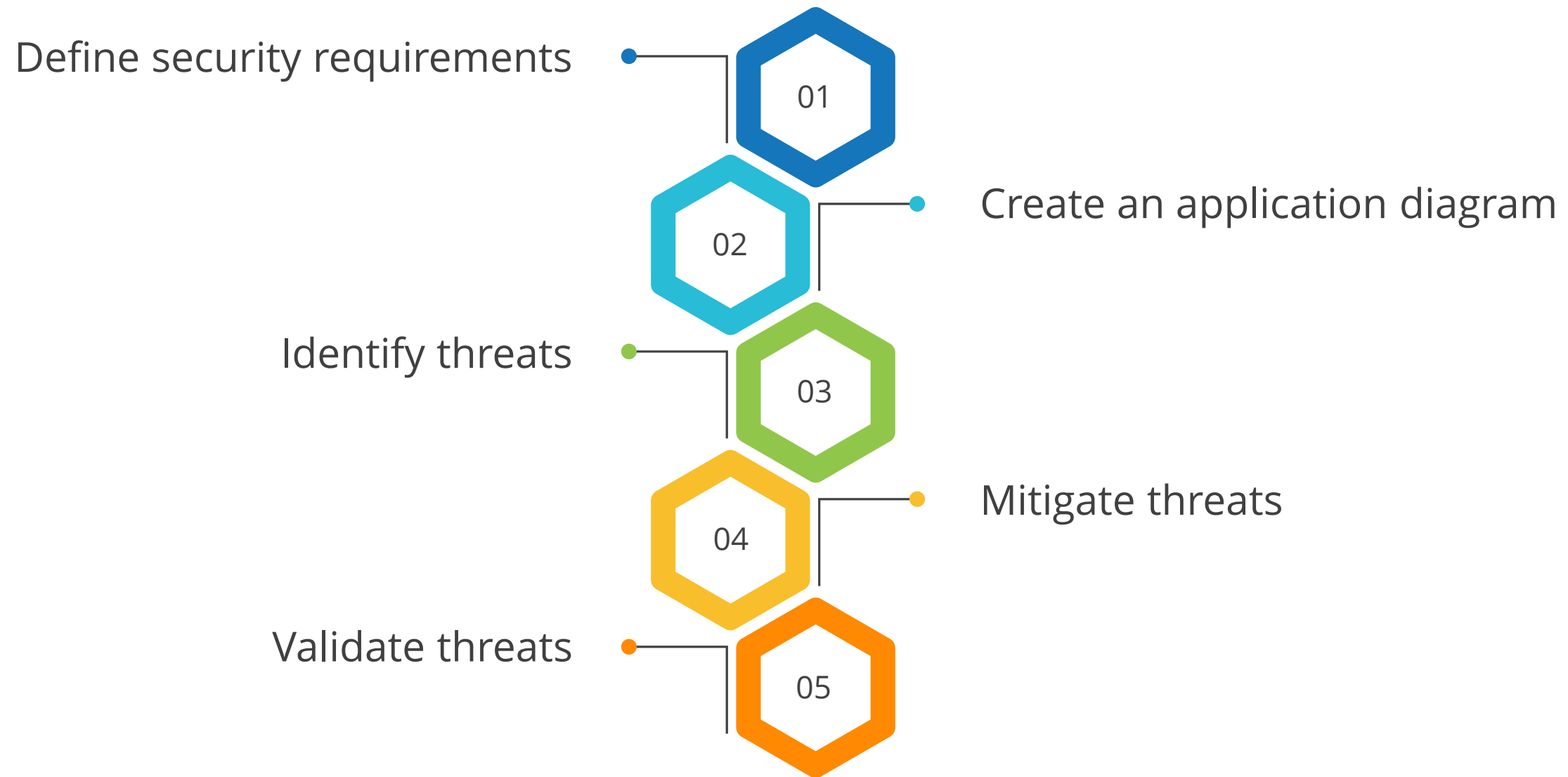
It supports long-term security goals and system resilience.

# Benefits of Threat Modeling in DevSecOps

- **Early identification of vulnerabilities:** It finds security issues during the design phase to avoid costly fixes later.
- **Risk prioritization:** It focuses on the most serious threats to address them first.
- **Improved collaboration:** It brings developers, security teams, and others together for a stronger security approach.
- **Cost-effective security measures:** It catches problems early, allowing cheaper fixes and reducing risks.

# Process of Threat Modeling

The threat modeling process involves the following five key steps:



# Process of Threat Modeling

## Defining security requirements

It uses threat modeling techniques to analyze the design and uncover potential security vulnerabilities.

## Creating an application diagram

It implements appropriate security controls and design changes to effectively address identified threats.

## Identifying threats

It thoroughly tests and validates that all identified threats have been mitigated, ensuring the system is secure.

# Process of Threat Modeling

## Mitigating threats

It implements appropriate security controls and design changes to effectively address identified threats.

## Validating threats mitigation

It thoroughly tests and validates that all identified threats have been mitigated, ensuring the system is secure.

# Tools for Threat Modeling

These are software applications that streamline the process of creating, analyzing, and visualizing threat models, allowing developers to efficiently identify, assess, and mitigate potential vulnerabilities.

Some of the common tools are as follows:



Microsoft thread modeling tool



IriusRisk Community Edition



CAIRIS



Thread Modeler



OWASP Thread Dragon



AWS ThreatComposer

# Tools for Threat Modeling



Microsoft thread modeling tool

It is a widely used tool provided by Microsoft to help users identify threats during the design phase of a system by using data flow diagrams.



IriusRisk Community Edition

It is an open-source tool designed for creating automated threat models and providing actionable security advice.

# Tools for Threat Modeling

The logo for CAIRIS, featuring the word "CAIRIS" in white, bold, uppercase letters on a dark blue rectangular background.

CAIRIS

It is a tool that combines risk analysis with requirements engineering to build threat models effectively.



Thread Modeler

It is a specialized tool that focuses on visualizing and managing threats in complex systems.

# Tools for Threat Modeling



OWASP Thread Dragon

It is an open-source threat modeling tool designed to integrate seamlessly with security and development workflows.



AWS ThreatComposer

It is a tool by AWS for creating and maintaining structured threat models tailored to cloud-native environments.

# Threat Modeling for Banking and Finance



## Banking and finance

- Banks implement threat modeling to secure critical financial applications and customer data.
- By proactively identifying risks, they protect against fraud, phishing attacks, and unauthorized transactions while ensuring compliance with regulatory standards.

## Quick Check

A financial institution is developing a new online banking application and wants to ensure the security of sensitive customer data and transactions. The team is using threat modeling to identify potential security risks. Which of the following actions best aligns with effective threat modeling?

- A. Testing security after launch
- B. Using STRIDE to find risks early
- C. Relying only on firewalls
- D. Skipping risk identification

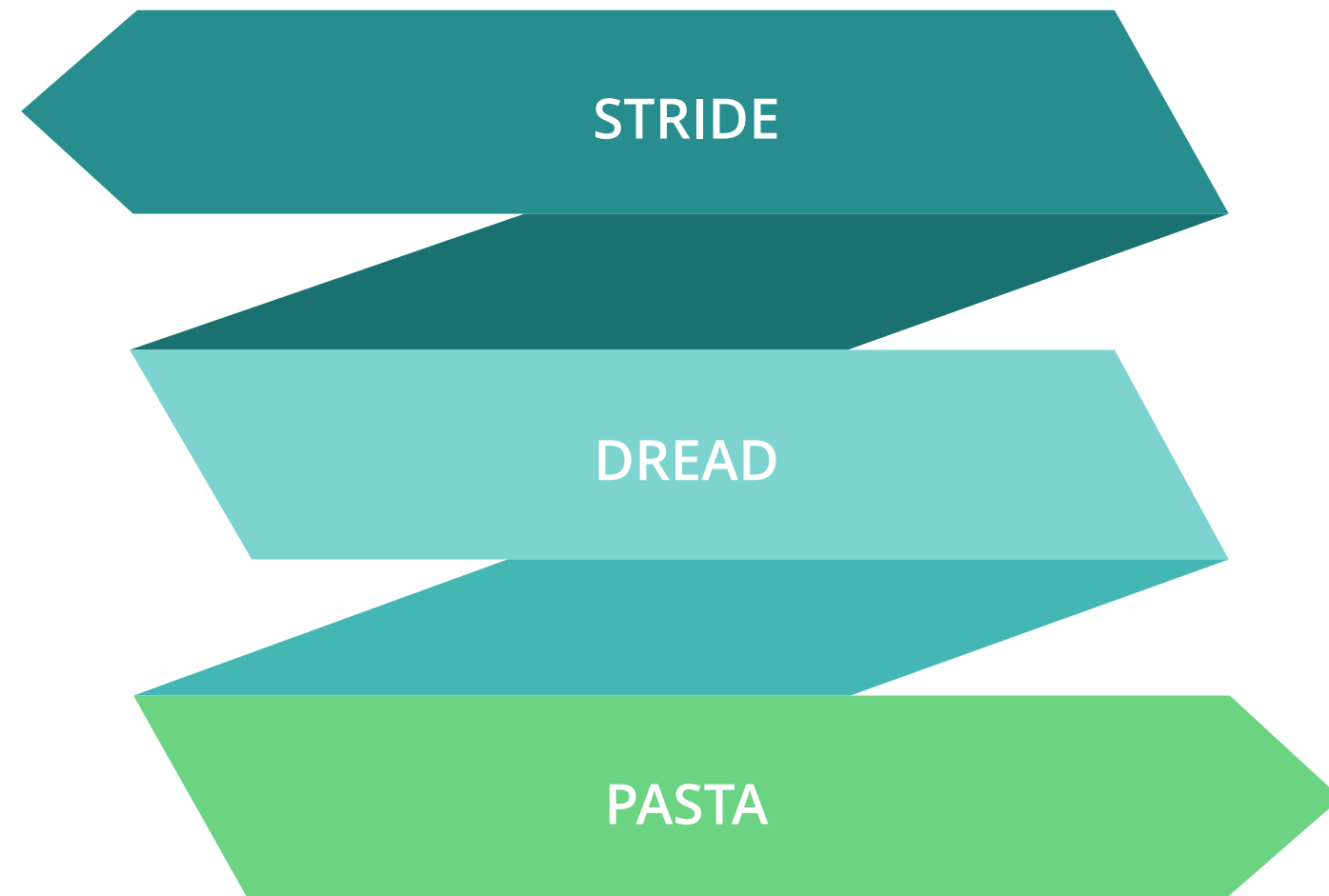




## Threat Modeling Techniques: Overview

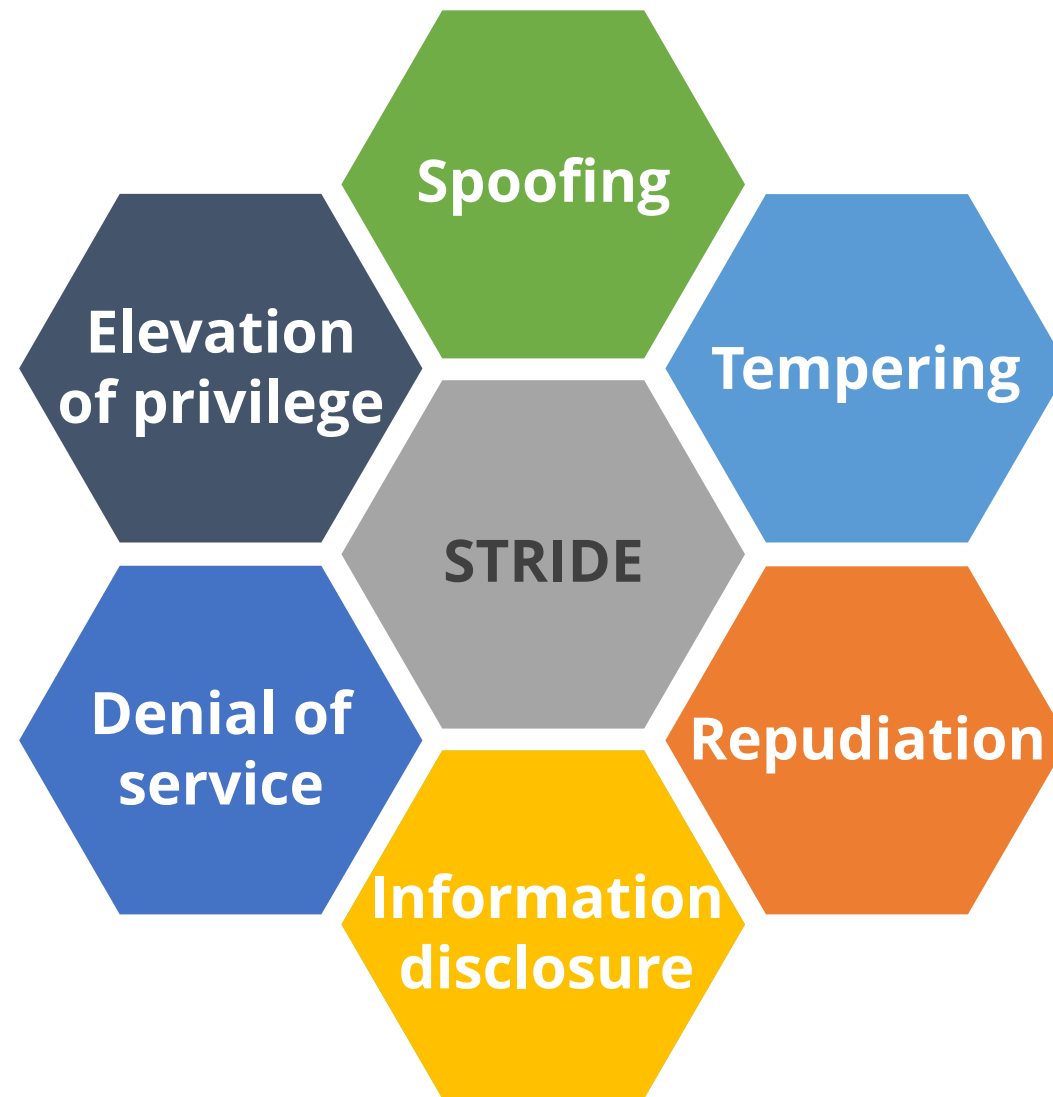
# Threat Modeling Techniques

The commonly used threat modeling techniques are as follows:



# What Is STRIDE?

It is a framework that identifies security threats which can be listed as follows:



# Key Components of STRIDE

It is a threat modeling technique that categorizes potential threats into six key areas:

## Spoofting

It is the act of impersonating another user or entity to gain unauthorized access.

## Tampering

It is the unauthorized alteration of data or systems,

## Repudiation

It is the ability to deny performing an action or transaction without proof to the contrary.

# Key Components of STRIDE

It is a threat modeling technique that categorizes potential threats into six key areas:

## Information disclosure

It is the exposure of confidential or sensitive information to unauthorized parties.

## Denial of service (DoS)

It is the act of disrupting the availability of a service, making it inaccessible to users.

## Elevation of privilege

It is gaining higher-level access than permitted, typically to perform unauthorized actions. [er to the demo document for detailed steps.](#)

# STRIDE for E-commerce Payment Gateway Security

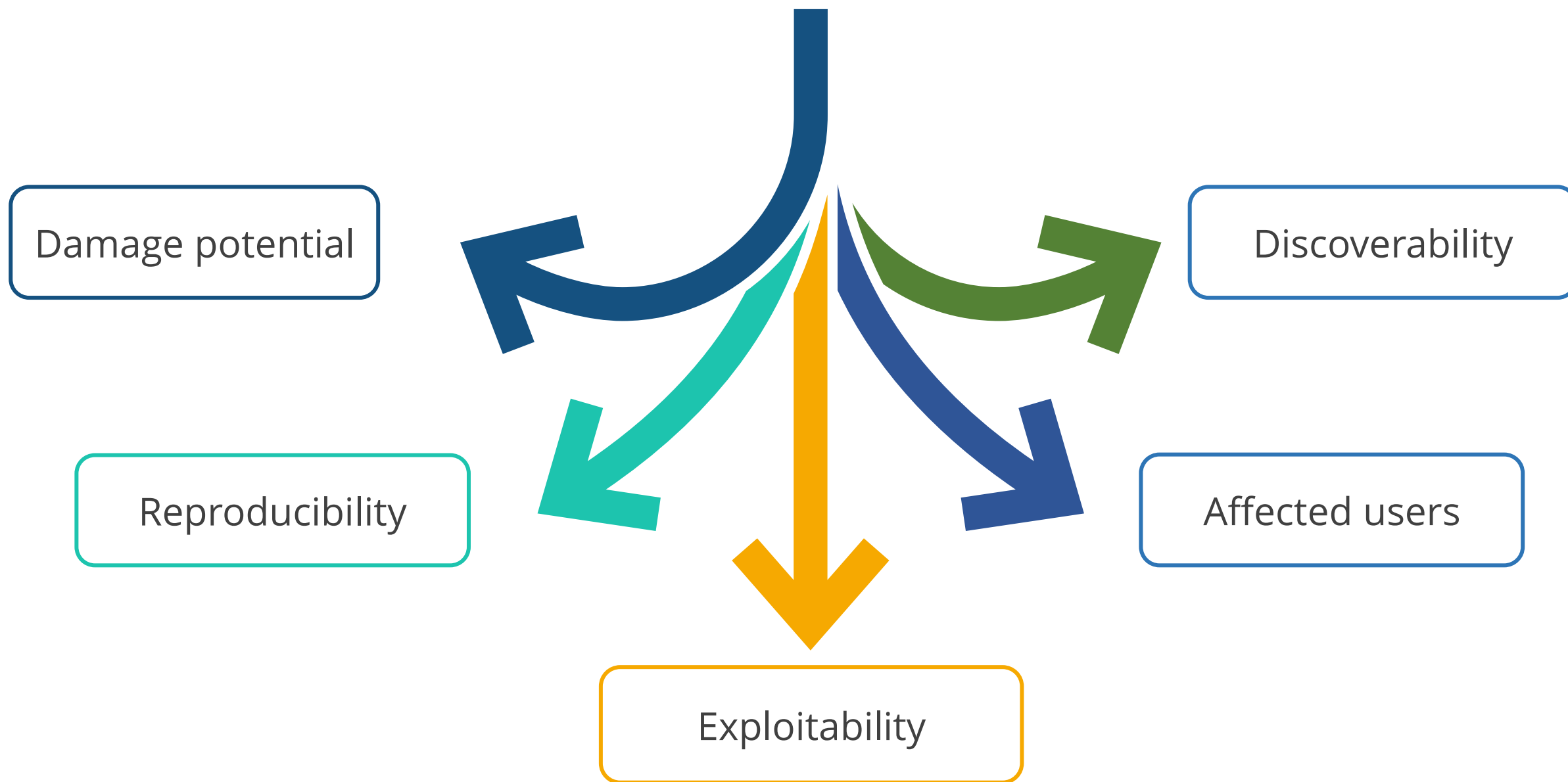


**E-commerce**

- E-commerce platforms implement STRIDE to secure payment gateways, which handle sensitive customer data like card details.
- By proactively identifying threats like spoofing, tampering, and information disclosure, they implement robust encryption protocols, multi-factor authentication, and activity monitoring.
- This ensures secure transactions, builds customer trust, and reduces the risk of fraudulent activities.

# What Is DREAD?

It is a model that ranks threats based on the following aspects:



# Key Factors of the DREAD Risk Assessment Model

It is a risk assessment methodology that prioritizes threats based on five factors:

**Damage potential:** It is the measure of how much harm the threat can cause to the system or users.

**Reproducibility:** It refers to the ease with which an attacker can consistently exploit a vulnerability.

**Exploitability:** It indicates the level of difficulty for an attacker to successfully exploit a vulnerability.

# Key Factors of the DREAD Risk Assessment Model

It is a risk assessment methodology that prioritizes threats based on five factors:

**Affected users:** It is the number of users who may be impacted by the threat.

**Discoverability:** It is the likelihood that the vulnerability can be discovered by an attacker.

# DREAD for E-commerce security



**E-commerce**

- An e-commerce platform uses the DREAD framework to assess risks like SQL injection and cross-site scripting.
- It evaluates threats based on damage, reproducibility, exploitability, affected users, and discoverability to prioritize security measures.
- This ensures customer data protection and reduces potential financial losses.

# What Is PASTA?

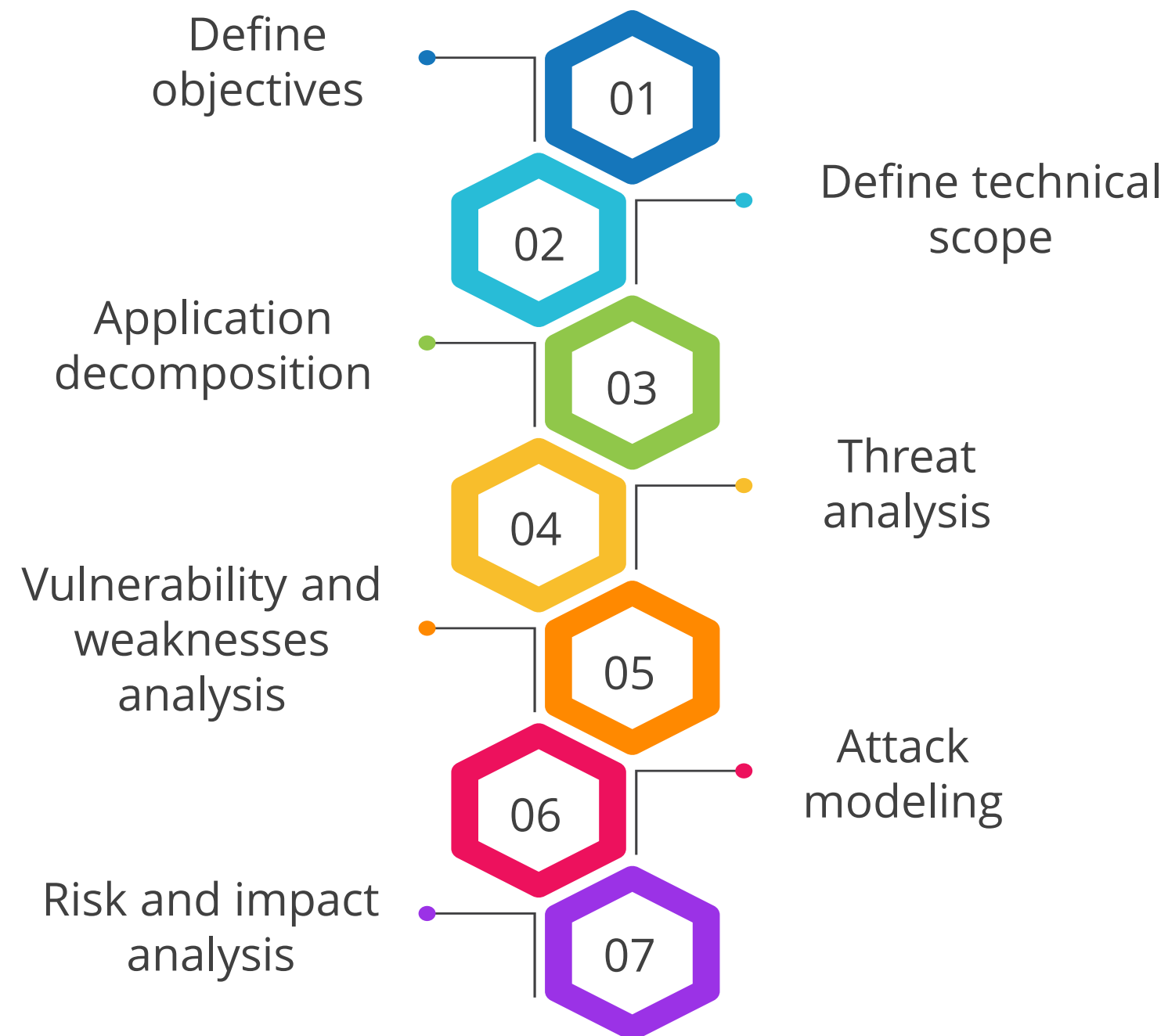
PASTA, also known as a Process for Attack Simulation and Threat Analysis, is a threat modeling technique that has the following factors:

**Attack scenarios:** It focuses on analyzing how an attacker might exploit vulnerabilities.

**Risk prioritization:** It is the process of assessing and ranking threats based on their likelihood and impact, guiding teams to address the most significant risks first.

# 7-Step Approach of PASTA

PASTA provides a 7-step process for risk analysis:



# 7-Step Approach of PASTA

- **Define objectives:** Establish the security goals and key assets requiring protection
- **Define technical scope:** Outline the system components and boundaries for threat analysis
- **Decompose application:** Break the application into smaller parts to understand its functionality
- **Analyze threats:** Identify potential threats and attack vectors targeting the system

# 7-Step Approach of PASTA

- **Assess vulnerabilities and weaknesses:** Evaluate the system for exploitable security gaps
- **Model attacks:** Simulate real-world attack scenarios to gauge their potential impact
- **Analyze risk and impact:** Assess the likelihood and severity of threats to prioritize risks and mitigation efforts

# PASTA for Banking Application Security



## Banking and finance

- A banking application uses the PASTA (Process for Attack Simulation and Threat Analysis) framework to identify and mitigate threats like unauthorized fund transfers and phishing attacks.
- By simulating potential attack scenarios, the framework helps prioritize risks based on business impact, enabling the implementation of robust security measures, such as encryption, fraud detection systems, and multi-factor authentication, to protect customer transactions and sensitive data.

# STRIDE vs. DREAD vs. PASTA

## STRIDE

- It helps identify security vulnerabilities across different threat categories.
- **Example:** Identifying spoofing vulnerabilities in a login system

## DREAD

- It provides a structured scoring system to rank threats by severity.
- **Example:** Ranking a SQL injection vulnerability based on how easily it can be exploited

## PASTA

- It focuses on analyzing attack scenarios to prioritize high-risk threats.
- **Example:** Simulating a Denial of Service (DoS) attack on a web server to measure impact

## Assisted Practice



### Implementing DREAD Model

Duration: 10 Min.

#### Problem statement:

You have been tasked to analyze potential threats to a system using the DREAD model to assess risk levels effectively.

#### Outcome:

By the end of this demo, you will be able to apply the DREAD model to evaluate and prioritize risks based on Damage potential, Reproducibility, Exploitability, Affected users, and Discoverability.

**Note:** Refer to the demo document for detailed steps: 01\_Implementing\_DREAD\_Model

# Assisted Practice: Guidelines



Steps to be followed:

1. Implement the DREAD model using a Python script

## Assisted Practice



### Implementing PASTA Model

Duration: 10 Min.

#### Problem statement:

You have been tasked to assess and analyze security risks using the PASTA (Process for Attack Simulation and Threat Analysis) model to align threat management with business objectives.

#### Outcome:

By the end of this demo, you will be able to implement the PASTA model to identify potential attack vectors, simulate threat scenarios, and prioritize mitigations effectively.

**Note:** Refer to the demo document for detailed steps: 02\_Implementing\_PASTA\_Model

# Assisted Practice: Guidelines



Steps to be followed:

1. Implement the PASTA model using a Python script

## Quick Check



A company is choosing a threat modeling approach to secure its new financial application. The team is considering STRIDE, DREAD, and PASTA to identify and manage potential risks. Which approach would help the team simulate real-world attacks and prioritize risks based on their likelihood and impact?

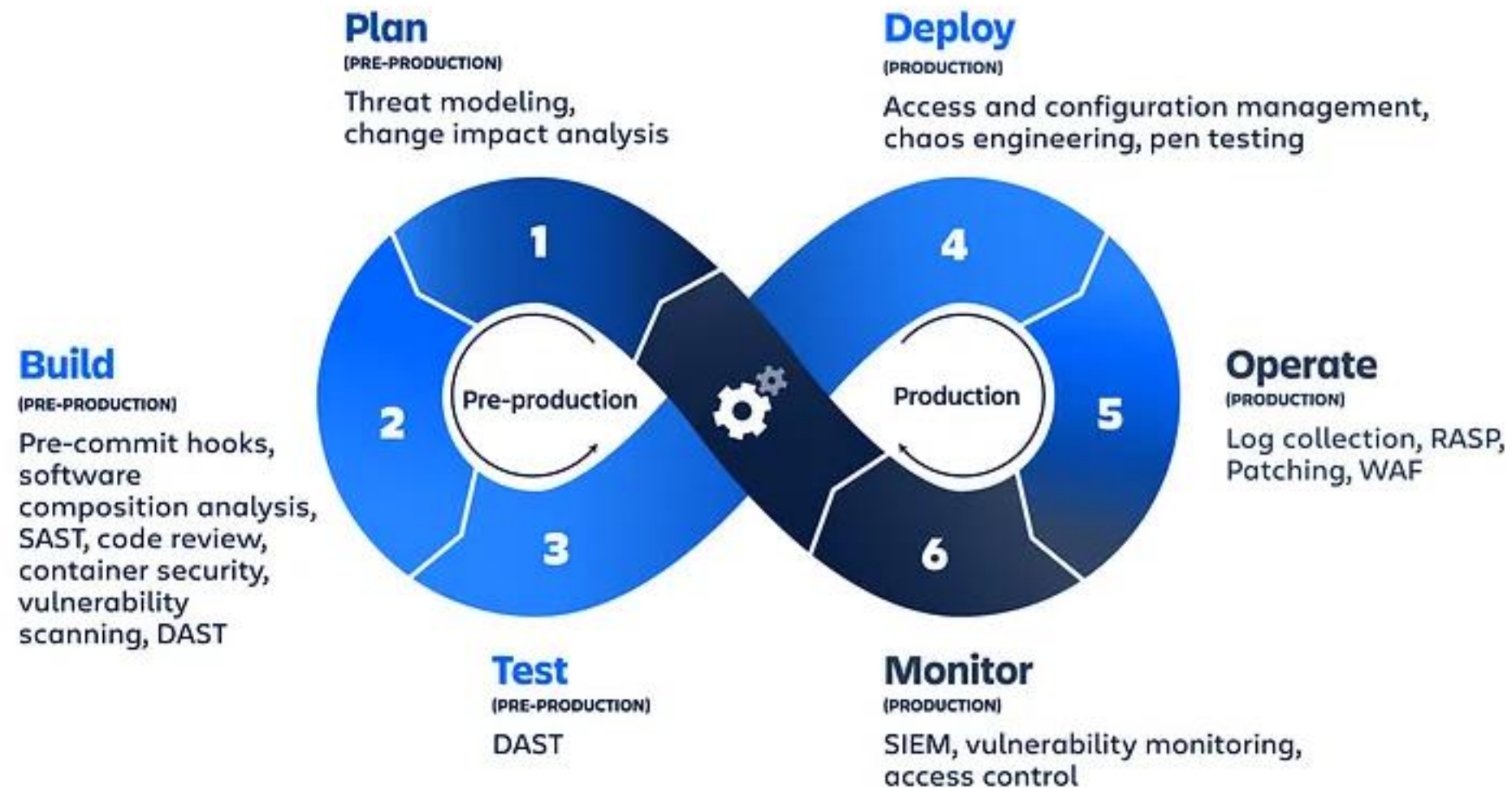
- A. STRIDE for categorizing threats
- B. DREAD for ranking risks
- C. PASTA for simulating attacks and analyzing impact
- D. Relying only on network security



# Securing DevSecOps Pipeline

# What Is a DevSecOps Pipeline?

It's an automated process that embeds security throughout the software development lifecycle, ensuring it is integral to each phase.



# DevSecOps Pipeline Phases

A DevSecOps pipeline integrates the following security phases throughout the software development lifecycle (SDLC), embedding security from initial design to production:

Plan (pre-production)

Involves threat modeling and change impact analysis to anticipate potential security issues and effects on the existing system

Build (pre-production)

Utilizes pre-commit hooks, software composition analysis, SAST, code review, container security, vulnerability scanning, and DAST to safeguard code quality and security pre-deployment

# DevSecOps Pipeline Phases

A DevSecOps pipeline integrates the following security phases throughout the software development lifecycle (SDLC), embedding security from initial design to production:

Test (pre-production)

Uses dynamic application security testing (DAST) to simulate real-world attacks and find vulnerabilities before the software goes live

Deploy (production)

Focuses on access and configuration management, chaos engineering, and penetration testing to manage deployments and maintain system integrity in production

# DevSecOps Pipeline Phases

A DevSecOps pipeline integrates the following security phases throughout the software development lifecycle (SDLC), embedding security from initial design to production:

Operate (production)

Includes log collection, runtime application self-protection (RASP), patching, and web application firewall (WAF) to safeguard operational software against new threats

Monitor (production)

Employs security information and event management (SIEM) and vulnerability monitoring, along with robust access control measures to continuously oversee and react to threats against the application infrastructure

# SAST, DAST, and SCA in DevSecOps Pipeline

In the DevSecOps pipeline, SAST, DAST, and SCA are integral tools that work concurrently to ensure security is embedded at every stage of the software development and deployment process, thus facilitating continuous security.



# SAST, DAST, and SCA in DevSecOps Pipeline

## Static Application Security Testing (SAST)

It analyzes source code or compiled versions of code during the build phase to detect security vulnerabilities early in the development lifecycle, before deployment. It ensures secure coding practices and reduces vulnerabilities at the initial stages.

## Dynamic Application Security Testing (DAST)

It evaluates the application during the testing phase by simulating runtime attacks on web applications to identify vulnerabilities visible to attackers. This approach tests the application's security posture under real-world conditions.

## Software Composition Analysis (SCA)

It identifies and analyzes third-party components in applications during the build phase. It ensures security and compliance by detecting vulnerabilities in external code, addressing risks associated with open-source dependencies.

# SAST: Features

## Early risk identification

Identifies security issues during the coding phase, reducing the cost and effort of fixing vulnerabilities later in development

## Compliance assurance

Ensures adherence to industry regulations and security standards by detecting code that fails to meet compliance requirements such as PCI-DSS or GDPR

## Shift-left security

Embeds security testing early in the SDLC, allowing developers to catch and fix vulnerabilities before the software moves to production

## SAST: Features

### Developer feedback

Provides real-time, actionable insights to developers, helping them address security risks during the coding process

### CI/CD integration

Scans code during the CI/CD process, ensuring continuous security validation without interrupting the development workflow

### Legacy code audits

Is used to scan older or legacy codebases, identifying vulnerabilities and ensuring that even outdated applications are secured against modern threats

## Tools for SAST



Scans code for security vulnerabilities with a focus on real-time feedback, integrating into developer workflows and emphasizing support for open-source vulnerability detection



Scans source code to identify security risks across multiple programming languages, integrating seamlessly into development workflows to ensure continuous monitoring



Inspects code continuously for quality and security vulnerabilities, integrating with CI/CD pipelines to automate analysis and providing detailed reports for developers

## Tools for SAST



Performs static analysis to detect vulnerabilities in code, offering detailed feedback with remediation guidance, and is designed for use by large and distributed teams



Detects critical defects and vulnerabilities within complex codebases, integrating into CI/CD environments to ensure ongoing security throughout the development process



Scans code to identify security flaws early in development, providing deep insights and extensive reporting, making it well-suited for large-scale enterprise projects

## Assisted Practice



### Running SAST Scan on a Static Web Application Using GitHub

Duration: 10 Min.

#### Problem statement:

You have been tasked to perform a Static Application Security Testing (SAST) scan on a static web application hosted on GitHub to identify potential vulnerabilities in the source code.

#### Outcome:

By the end of this demo, you will be able to configure and run a SAST scan on a static web application using GitHub, analyze the scan results, and implement recommendations to improve code security.

**Note:** Refer to the demo document for detailed steps:  
03\_Running\_SAST\_Scan\_on\_a\_Static\_Web\_Application\_Using\_GitHub

# Assisted Practice: Guidelines



Steps to be followed:

1. Fork the GitHub repository
2. Set up a workflow
3. Connect the GitHub account with Semgrep
4. Validate the SAST scan

# DAST: Features

## Input/Output validation

Identifies vulnerabilities related to improper handling of user inputs and outputs

## Authentication weaknesses

Detects flaws in authentication mechanisms by simulating real-world attacks

## Server configuration

Assesses security risks due to misconfigured servers in a live environment

# DAST: Features

## Simulating real-time attacks

Tests application response to dynamic attacks without accessing the source code

## Security gaps in applications

Targets live applications to uncover vulnerabilities that may be missed during static testing

## Tools for DAST



It is a widely used tool for detecting vulnerabilities in web applications by simulating real-world attacks. It is popular among developers and security teams for its extensive features and ease of use.



It is a platform that provides tools for web security testing, including both automated scans and manual options. Its professional edition adds advanced features like vulnerability management.



It is a scanner that automates the detection of vulnerabilities like SQL injection and XSS. It also provides detailed reports to help developers fix the identified issues quickly.

## Tools for DAST



It is a tool that offers highly accurate vulnerability detection for both web applications and APIs by leveraging its automation and proof-based scanning technology.



It is a cloud-based web application security scanner that automates the detection of a wide range of vulnerabilities. It is designed to scale easily, making it ideal for enterprises managing multiple apps.



It is a dynamic scanner that tests the security of web and mobile applications by simulating attacks and providing insights into potential security risks.

## Assisted Practice



### Performing DAST Scan on a Running Web Application with OWASP ZAP

Duration: 10 Min.

#### Problem statement:

You have been tasked to perform a Dynamic Application Security Testing (DAST) scan on a running web application using OWASP ZAP to identify security vulnerabilities in the application.

#### Outcome:

By the end of this demo, you will be able to configure and execute a DAST scan using OWASP ZAP, interpret the scan results, and propose security enhancements for the identified vulnerabilities.

**Note:** Refer to the demo document for detailed steps:

04\_Performing\_DAST\_Scan\_on\_a\_Running\_Web\_Application\_with\_OWASP\_Zap

# Assisted Practice: Guidelines



Steps to be followed:

1. Configure DAST tool with GitHub Actions

## SCA: Features

**Open-source risk management:** Detects and addresses security vulnerabilities in third-party components used in applications

**License compliance:** Ensures all open-source components adhere to legal licensing requirements

**Supply chain security:** Protects the software supply chain by monitoring dependencies for security issues

## Tools for SCA



Helps manage open-source security and compliance by identifying vulnerabilities and license risks in software components



Automates the detection of security vulnerabilities and licensing issues in open-source components throughout the development lifecycle



Provides open-source compliance and security by scanning for vulnerabilities and tracking license adherence in real time

# SAST vs. DAST vs. SCA

| Aspect           | SAST                                                    | DAST                                          | SCA                                                        |
|------------------|---------------------------------------------------------|-----------------------------------------------|------------------------------------------------------------|
| Testing approach | Analyzes source code or binaries without executing them | Tests the application while it's running      | Analyzes third-party components and libraries              |
| Type of testing  | White-box testing (internal view)                       | Black-box testing (external view)             | Component-based analysis                                   |
| Focus            | Code vulnerabilities (example: logic flaws)             | Application behavior in real-world conditions | Vulnerabilities and license compliance in third-party code |

SAST vs. DAST vs. SCA

| Aspect                 | SAST                                    | DAST                                  | SCA                                           |
|------------------------|-----------------------------------------|---------------------------------------|-----------------------------------------------|
| Detection timing       | Early in the development cycle          | During runtime or after deployment    | Anytime in the software lifecycle             |
| Common vulnerabilities | Logic errors, insecure coding practices | SQL injection, XSS, misconfigurations | Vulnerable or outdated open-source components |
| Access requirement     | Access to source code                   | No access to source code              | Requires knowledge of third-party libraries   |

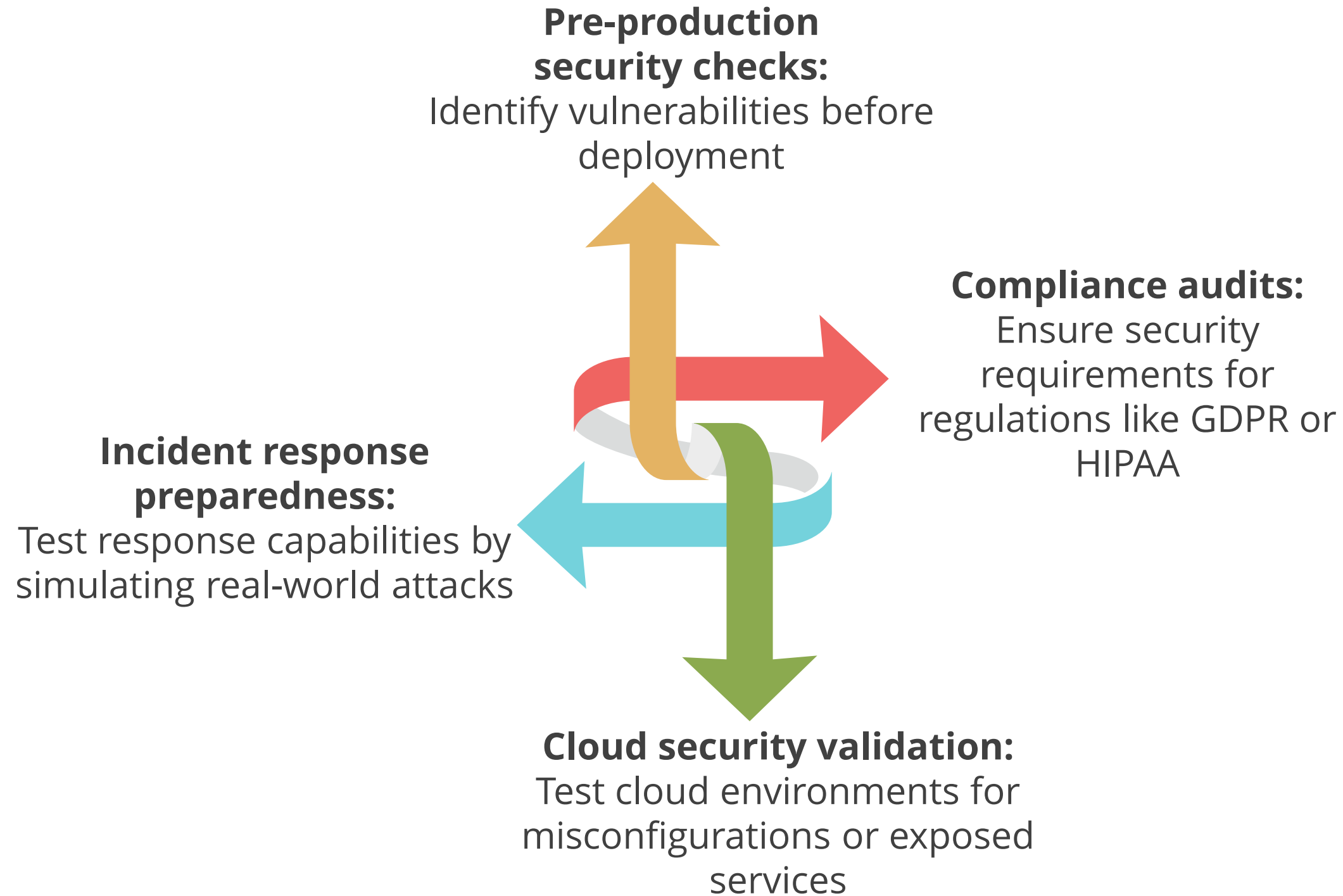
# What Is Penetration Testing?

Penetration testing, or pen testing, is crucial in DevSecOps. It involves authorized attempts to uncover security weaknesses in applications and infrastructure.



Regular implementation of pen testing helps in the identification and remediation of potential vulnerabilities.

# Penetration Testing: Features



# Types of Penetration Testing in DevSecOps

Penetration testing in DevSecOps is a security measure where ethical hackers attempt to exploit system vulnerabilities to identify potential risks. Various types include:

**White box testing:** Uses full system knowledge to uncover deep vulnerabilities

**Black box testing:** Tests systems with minimal information, simulating unknown threats

**Internal testing:** Mimics insider threats to find internal security gaps

**External testing:** Simulates attacks from outside the organization to test defenses

# What Is Vulnerability Management?

It is the continuous process of identifying, assessing, and addressing security weaknesses within systems to reduce risk and prevent potential exploits.

The following are the key elements while performing vulnerability management:

## Identification

Detects vulnerabilities using tools to scan code, dependencies, and settings

## Observability

Translates detected issues into insights that help teams prioritize risks effectively

## Management

Organizes and addresses vulnerabilities based on risk levels to implement fixes and reduce threats

# Vulnerability Management: Features

**Protecting sensitive data:** Prevents unauthorized access to critical information

**Ensuring compliance:** Ensures to meet standards like GDPR, HIPAA, and PCI-DSS

**Enabling early detection:** Identifies vulnerabilities before exploitation

**Maintaining continuous security:** Supports security in dynamic DevOps

**Prioritizing risks:** Allocates resources to address high-risk vulnerabilities first

# Secure Coding Techniques

Secure coding is essential for developing software protected against the prevalent threats and vulnerabilities that can lead to severe security breaches. Here are some pivotal techniques used in secure coding:

## **Input validation:**

Validate user inputs to prevent attacks like SQL injection and cross-site scripting (XSS)

## **Use of safe libraries:**

Implement trusted and tested libraries to avoid introducing vulnerabilities

## **Error handling:**

Ensure errors are handled properly without exposing sensitive system information

# Secure Coding Techniques

Here are some pivotal techniques used in secure coding:

## **Authentication and authorization:**

Enforce strong authentication and proper role-based access control (RBAC) for secure user sessions

## **Code reviews:**

Regularly conduct peer reviews to identify and fix potential security issues

## **Encryption:**

Use encryption to protect sensitive data at rest and in transit

## Assisted Practice



### Performing Penetration Testing Using OWASP ZAP and GitHub Actions

Duration: 10 Min.

#### Problem statement:

You have been tasked to perform penetration testing on a web application using OWASP ZAP integrated with GitHub Actions to automate the testing process in a CI/CD pipeline.

#### Outcome:

By the end of this demo, you will be able to configure OWASP ZAP with GitHub Actions, execute automated penetration tests, and interpret the results to enhance the security of your web application.

**Note:** Refer to the demo document for detailed steps:  
[05\\_Performing\\_Penetration\\_Testing\\_Using\\_OWASP\\_ZAP\\_and\\_GitHub\\_Actions](#)

# Assisted Practice: Guidelines



Steps to be followed:

1. Configure DAST tool with GitHub Actions

## Quick Check

A company is expanding its cloud infrastructure, increasing complexity and potential attack vectors. Which security measure should be prioritized to secure its systems against external threats?

- A. White box testing to utilize extensive system knowledge and conduct an exhaustive vulnerability search
- B. Black box testing to see how the system handles unexpected external threats
- C. Internal testing to detect potential security breaches from within the organization
- D. External testing to test the organization's external defense capabilities





## **Debugging in DevSecOps**

# What Is Debugging?

It is the process of finding, diagnosing, and fixing errors or issues in software or systems to ensure they work properly.



It involves reviewing code, configurations, or processes to identify the root cause of problems and apply solutions.

# What Is Debugging in DevSecOps?

It is the practice of identifying, diagnosing, and fixing security and operational issues within the integrated development, security, and operations pipeline.



Debugging in DevSecOps helps create a seamless and secure development environment by addressing issues early and continuously throughout the development.

# Importance of Debugging in DevSecOps

## **Ensures secure code**

Debugging identifies and fixes security issues before deployment.

## **Improves monitoring**

It helps catch security issues during development.

## **Reduces failures**

It prevents errors that could cause security problems in production.

## **Supports compliance**

It helps meet security rules and best practices.

# Debugging Strategies for Secure Pipelines

The following are effective debugging strategies to strengthen pipeline security:

Integrate automated testing

Run automated security tests to catch vulnerabilities early in the pipeline

Use static code analysis

Scan code with static analysis tools to find issues before runtime

Implement continuous monitoring

Monitor the pipeline in real time to detect and respond to threats quickly

Enable detailed logging

Use detailed logging to trace and resolve issues during build and deployment

# Tools for Debugging in DevSecOps

These are some commonly used tools for effective debugging in DevSecOps to maintain secure and efficient development pipelines:

Some of the common tools are as follows:



SonarQube



OWASP ZAP (Zed Attack Proxy)



Splunk



Grafana loki



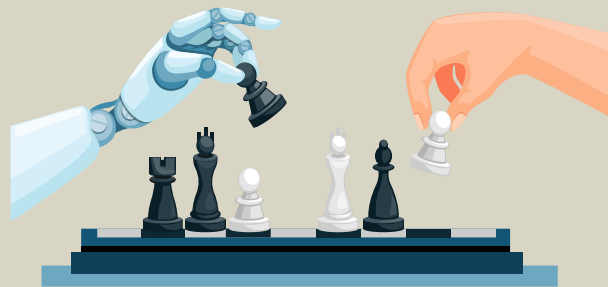
Datadog



Prometheus

# Use Case

## DevSecOps Implementation for SaaS Deployment Optimization at SynCore



### Problem

- SynCore, a SaaS company, faced challenges with a largely manual deployment process using Bitbucket for DevOps.
- This included inconvenient updates on virtual machines without proper version control or packaging, security vulnerabilities from exposed access keys, inefficient unit testing methods, and limited ability to deploy UI files via a CDN.
- The process was time-consuming, error-prone, and lacked scalability.

# Use Case

## DevSecOps Implementation for SaaS Deployment Optimization at SynCore



### Solution

The company implemented a comprehensive DevSecOps framework to streamline and secure their workflows. Key solutions included:

- CI/CD pipelines were established using Azure DevOps and Firebase Hosting for managing front-end components.
- Static code analysis tools such as SonarQube were integrated to identify vulnerabilities early in the development process.

# Use Case

## DevSecOps Implementation for SaaS Deployment Optimization at SynCore



### Solution

- Automated pipelines were set up for unit and penetration tests to ensure consistent testing of code quality.
- Continuous vulnerability scans were conducted with OWASP-recommended tools to identify and mitigate risks.
- Compliance monitoring was implemented to provide real-time alerts on security setting changes.
- Deployment processes were optimized to enable efficient packaging and management of software updates.

# Use Case

## DevSecOps Implementation for SaaS Deployment Optimization at SynCore



### Outcome

The DevSecOps transformation delivered significant improvements:

- Deployment time reduced by 57%, enabling over 10 builds per month instead of 4.
- Over 95% of bugs were caught before production, minimizing user-reported issues from 20 to just 1-2 per month.

# Use Case

## DevSecOps Implementation for SaaS Deployment Optimization at SynCore



### Outcome

- Security incidents dropped by 82%, while vulnerabilities decreased from 10 to 2-3 per quarter.
- Productivity increased, saving 40+ hours per month through automation.
- System uptime improved to over 99%, while downtime reduced to less than 7 hours per month.

## Quick Check



A DevSecOps team is integrating debugging practices to catch and fix security issues during the development phase of their CI/CD pipeline. What outcome will this most likely lead to in their deployment process?

- A. Increase hardware usage
- B. Secure code before deployment
- C. Expedite time to market
- D. Reduce team collaboration

# Key Takeaways

- DevSecOps integrates security into each phase of development, ensuring faster and safer software delivery.
- Threat modeling frameworks like STRIDE and DREAD provide structured ways to identify and mitigate potential risks.
- A DevSecOps pipeline incorporates security measures from design to deployment, facilitating a secure development environment.
- OWASP ZAP is a widely used tool for detecting vulnerabilities in web applications by simulating real-world attacks.
- Penetration testing, or pen testing, is crucial in DevSecOps, involving authorized attempts to uncover security weaknesses in applications and infrastructure.
- Debugging is the process of finding, diagnosing, and fixing errors or issues in software or systems to ensure they work properly.



# Integrating Penetration Testing and SQL Injection in DevSecOps with GitHub Actions

**Duration: 25 Min.**

**Project agenda:** To implement penetration testing and SQL injection detection in a DevSecOps pipeline using GitHub Actions, enhancing the security of a Java application by automating security scans and generating vulnerability reports

**Description:** As a DevOps engineer, your task is to enhance the security of a deliberately vulnerable application by incorporating penetration testing and SQL injection detection into a GitHub Actions workflow. This involves setting up workflows to automatically perform penetration scans using OWASP ZAP, detect vulnerabilities, and log results as GitHub issues, ensuring early identification of potential threats.



# Integrating Penetration Testing and SQL Injection in DevSecOps with GitHub Actions

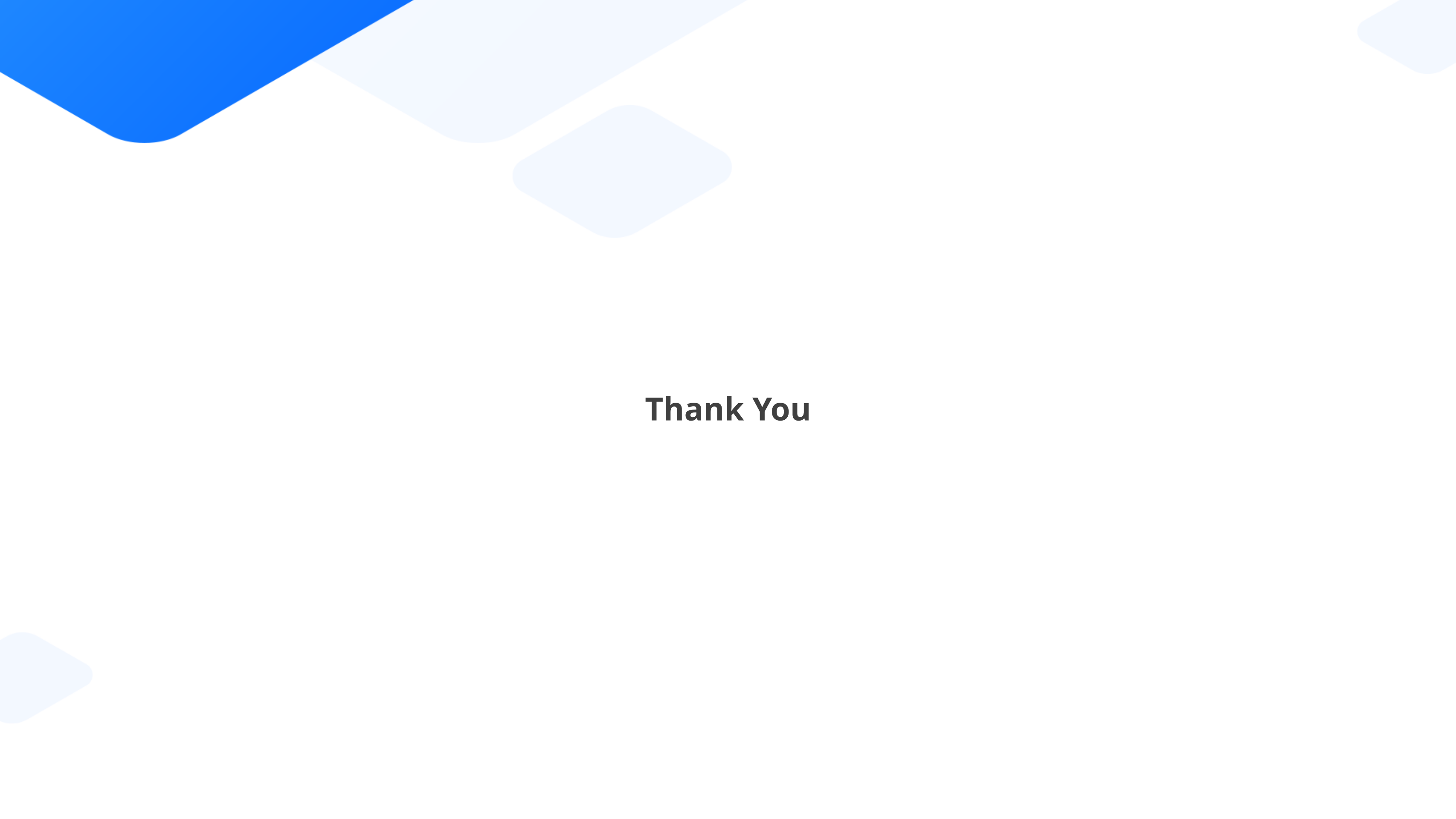
Duration: 25 Min.

## Perform the following:

1. Set up EC2 Instance
2. Deploy the vulnerable application
3. Test SQL Injection
4. Create GitHub repository and set up Actions Workflow
5. Run Workflow and review results

**Expected deliverables:** A GitHub Actions workflow for automated penetration testing using OWASP ZAP, a deployed WebGoat application on the EC2 instance, a baseline scan report identifying vulnerabilities like SQL injection, GitHub issues logged for detected vulnerabilities, and a consolidated security report in the repository





**Thank You**